



F107 – Foundations of Computer Science

Lecture 7

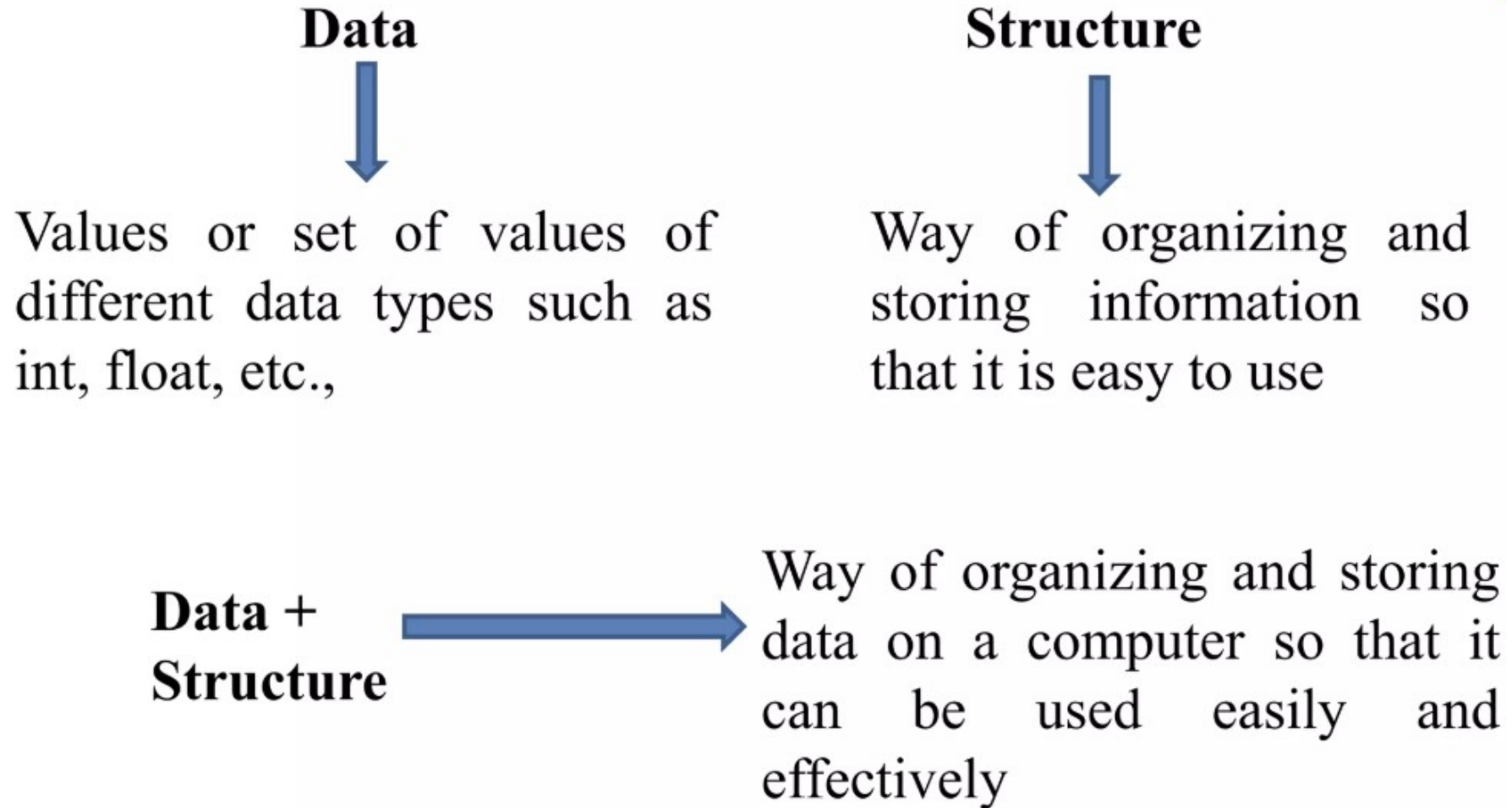
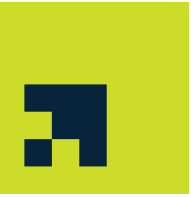
Dr. Sanjit Kumar Saha

Topics



- Data Structures
 - Arrays
 - Records
 - Linked Lists
- Algorithms
 - Basic Constructs
 - Sequence, Selection, Repetition
 - Sorting and Searching
 - Sub-Algorithms
 - Iteration vs. Recursion
- Topics covered in Book in Chapter 8 and 11 (additional info on UML in Appendix B*)

Data + Structure

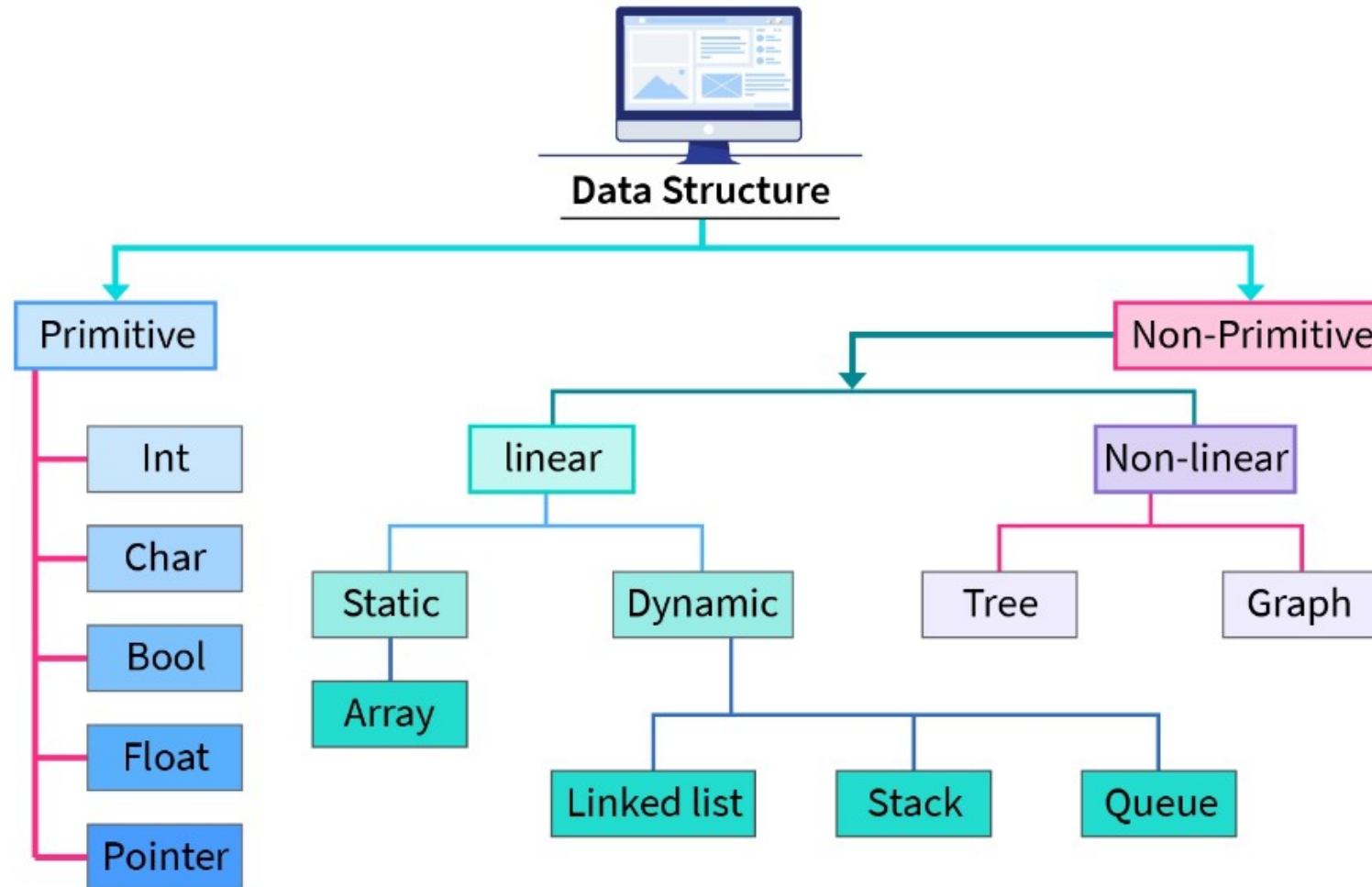
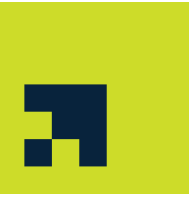


Data Structure

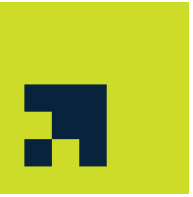


- Data Structure is defined as a group of data elements provides an efficient way of storing or organizing data in a computer.
- It is used in almost every program or software system.
- Examples: Arrays, Linked-list, Stack, Queue, Tree, Graph

Classifications of Data Structure



Data Structure Operations

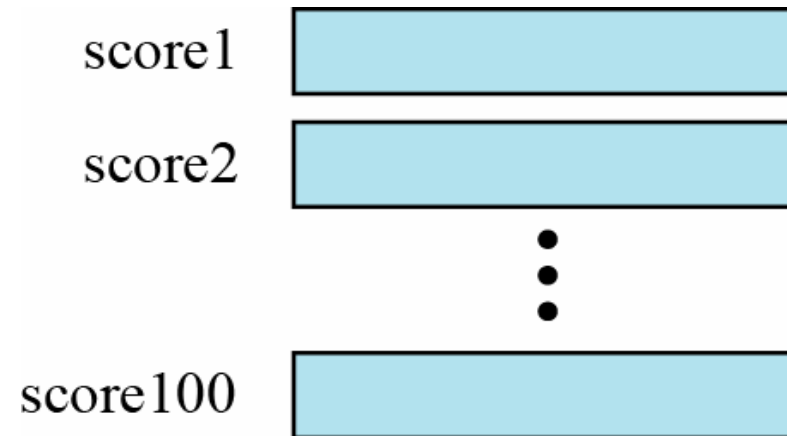


- Four basic operations:
 - *Traversing*: Accessing each record exactly once so that certain items in the record may be processed.
 - *Searching*: Finding the location of the record or finding the location of all records, which satisfy one or more conditions.
 - *Inserting*: Adding new records to the structure.
 - *Deleting*: Removing a record from the structure.
- The following two operations are used in special situations:
 - *Sorting*: Arranging the records in some logical orders.
 - *Merging*: Combining the records in two different sorted files into a single file.

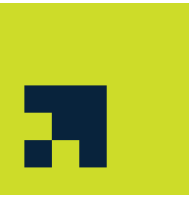
ARRAYS



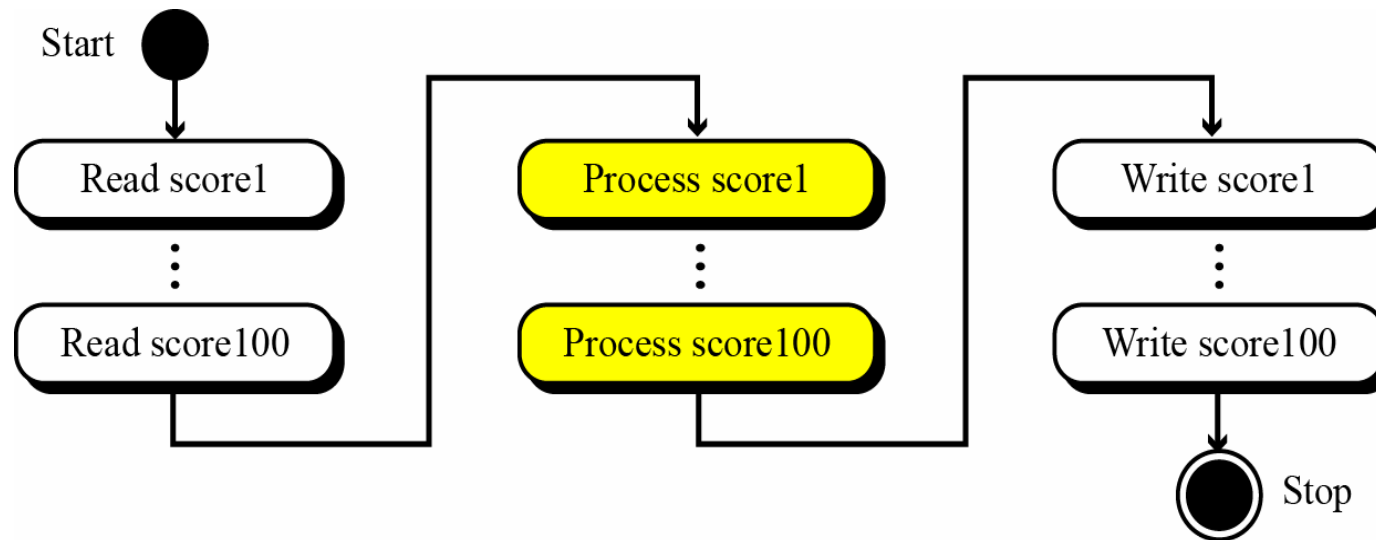
Imagine that we have 100 scores. We need to read them, process them and print them. We must also keep these 100 scores in memory for the duration of the program. We can define a hundred variables, each with a different name, as shown in Figure.



A hundred individual variables

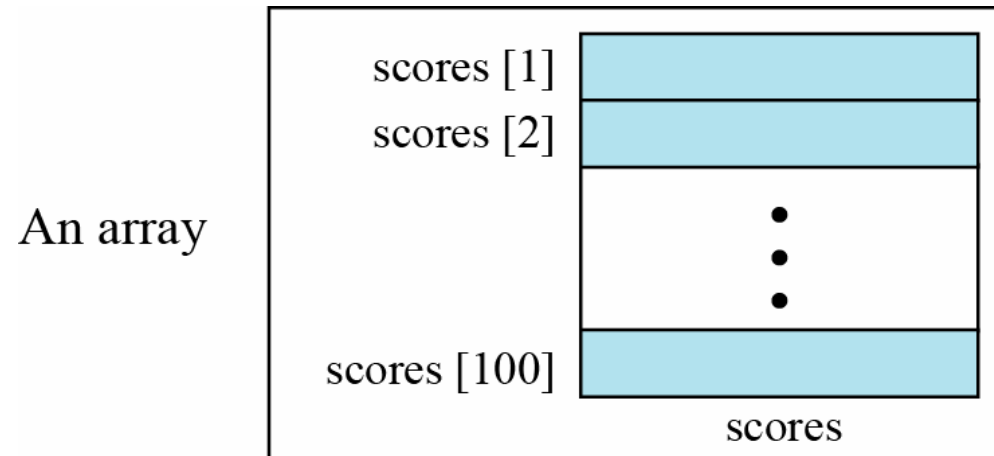


But having 100 different names creates other problems. We need 100 references to read them, 100 references to process them and 100 references to write them. Figure 11.2 shows a diagram that illustrates this problem.



Processing individual variables

- An array is a sequenced collection of elements, normally of the same data type, although some programming languages accept arrays in which elements are of different types.
- We can refer to the elements in the array as the first element, the second element and so forth, until we get to the last element.

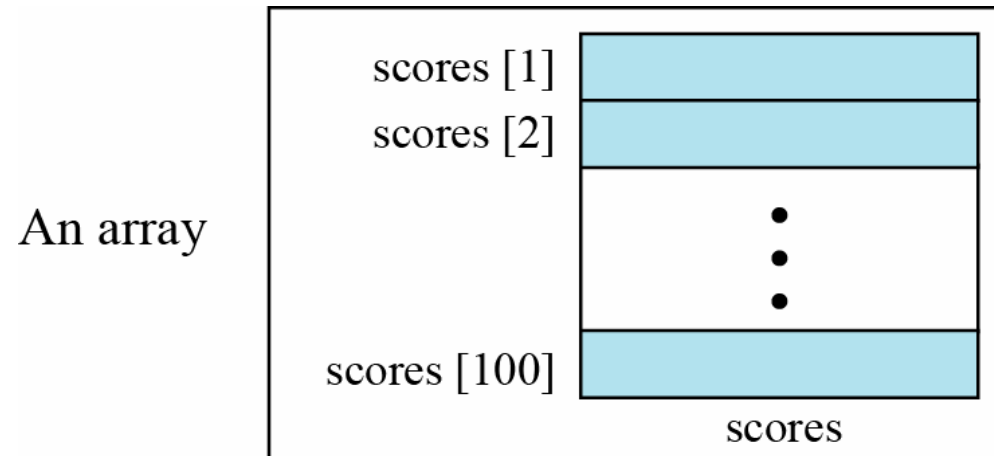


Arrays with indexes

Array name versus element name



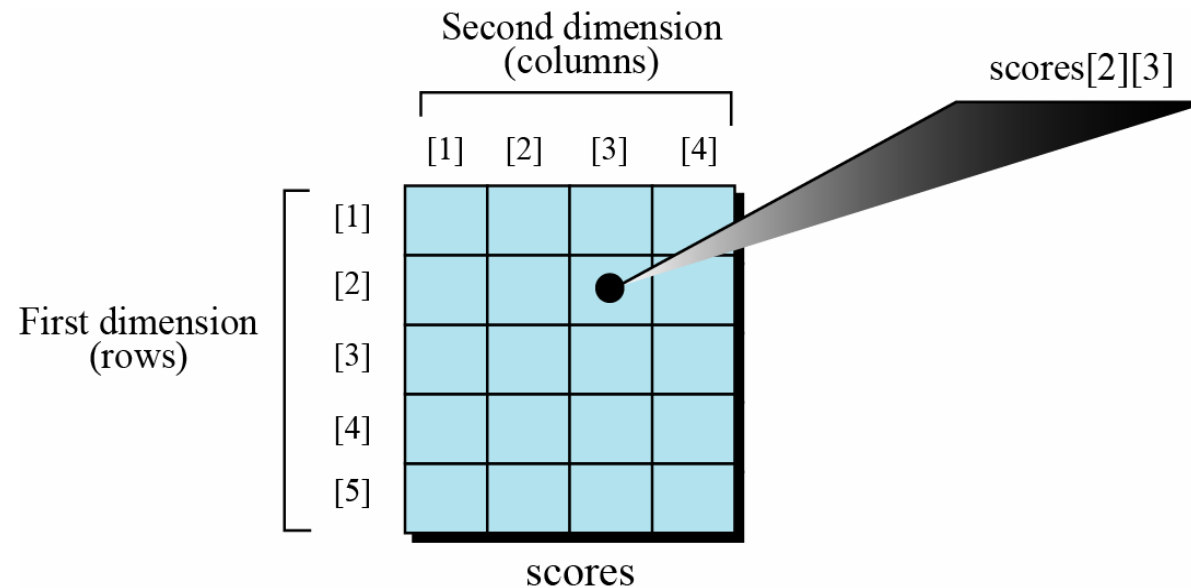
- In array, we have two types of identifiers:
- the name of the array - scores
- the name of each individual element - scores[1], scores[2], and so on.



Multi-dimensional arrays



- Many applications require that data be stored in more than one dimension.
- Figure shows a table, which is commonly called a **two-dimensional** array.



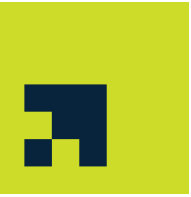
A two-dimensional array

Operations on array



- Common operations on arrays as structures are:
 - *searching*
 - *insertion*
 - *deletion*
 - *retrieval*
 - *traversal*
- Although searching, *retrieval* and *traversal* of an array is an easy job, *insertion* and *deletion* is time consuming.
- The elements need to be *shifted down* before *insertion* and *shifted up* after *deletion*.

Application



- If we have a list in which a lot of insertions and deletions are expected after the original list has been created, we should not use an array.
- An array is more suitable when the number of deletions and insertions is small, but a lot of searching and retrieval activities are expected.

An array is a suitable structure when a small number of insertions and deletions are required, but a lot of searching and retrieval is needed.

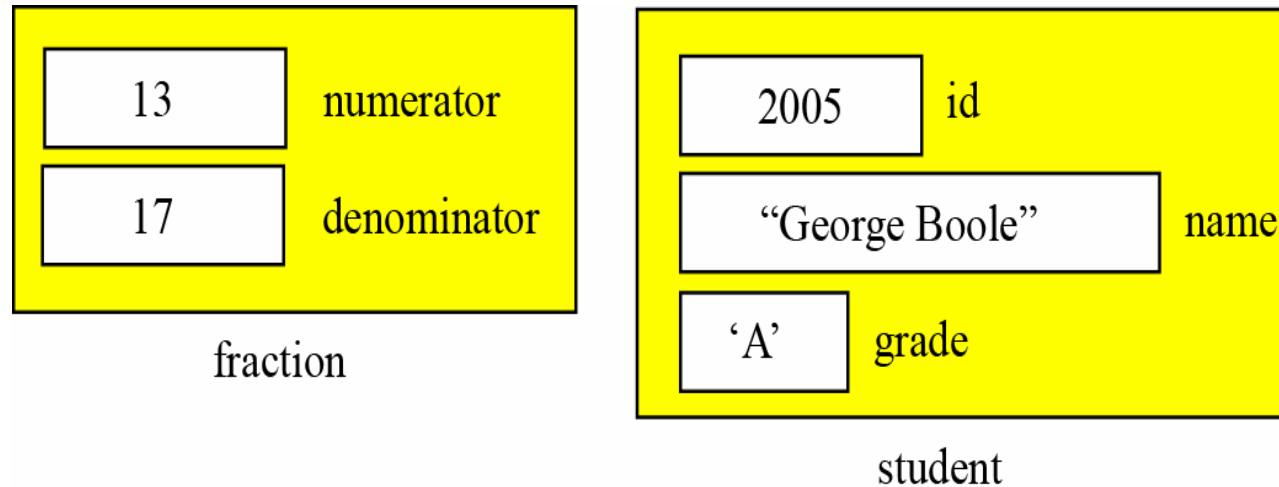
RECORDS



- A *record* is a collection of related elements, possibly of different types, having a single name.
- Each element in a record is called a *field*.
- A *field* is the smallest element of named data that has meaning.
- A *field* has a type and exists in memory.
- *Fields* can be assigned values, which in turn can be accessed for selection or manipulation.
- A *field* differs from a variable primarily in that it is part of a record.

Examples

- The first example, fraction, has two fields, both of which are integers.
- The second example, student, has three fields made up of three different types.

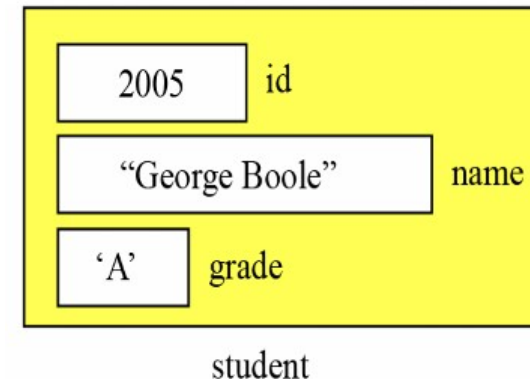


Records

Record name *versus* field name



- Just like in an array, we have two types of identifier in a record:
 - the name of the record
 - the name of each individual field inside the record
- The name of the record is the name of the whole structure, while the name of each field allows us to refer to that field.
- For example, in the student record, the name of the record is student, the name of the fields are **student.id**, **student.name** and **student.grade**.
- Most programming languages use a *period* (.) to separate the name of the structure (record) from the name of its components (fields).

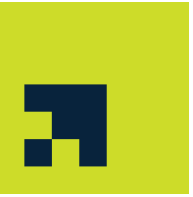


Comparison of records and arrays

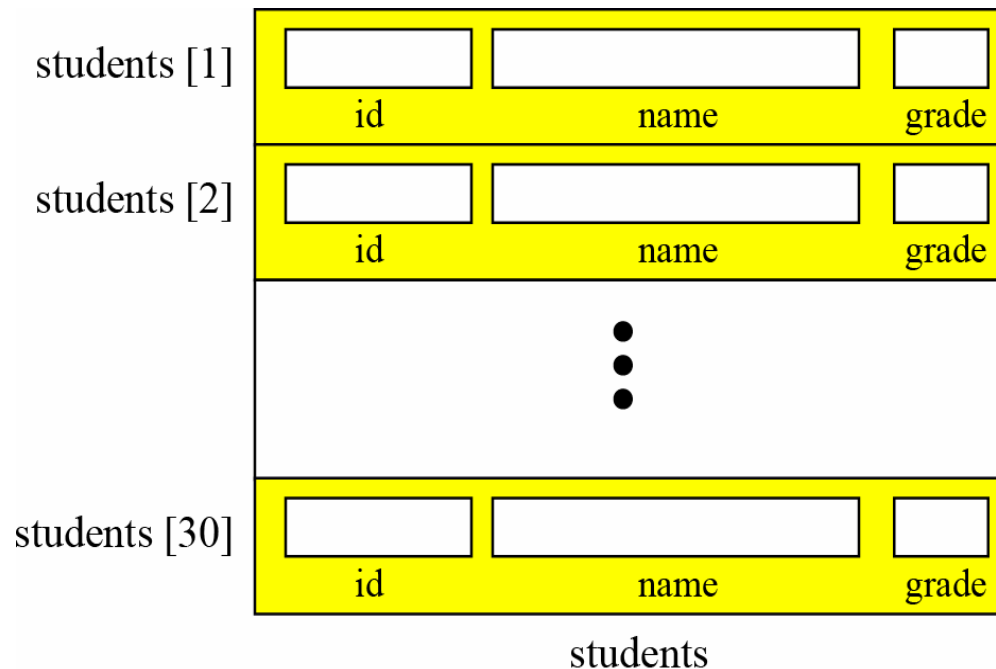


- An array defines a combination of elements, while a record defines the identifiable parts of an element.
- For example, an array can define a class of students (40 students), but a record defines different attributes of a student, such as id, name or grade.

Array of records



- If we need to define a combination of elements and at the same time some attributes of each element, we can use an array of records.
- For example, in a class of 30 students, we can have an array of 30 records, each record representing a student.



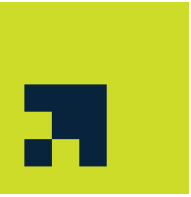
Array of records

Arrays versus arrays of records



- Both an array and an array of records represent a list of items.
- An array can be thought of as a special case of an array of records in which each element is a record with only a single field.

LINKED LISTS



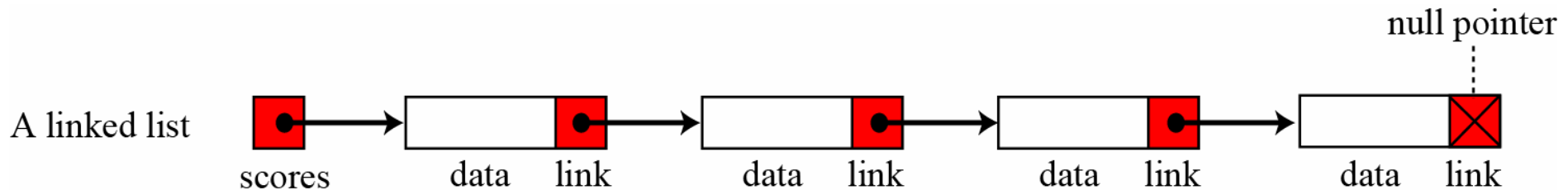
- A *linked list* is a collection of data in which each element contains the location of the next element—that is, each element contains two parts:

- *data*

- *link*

- The name of the list is the same as the name of this pointer variable.

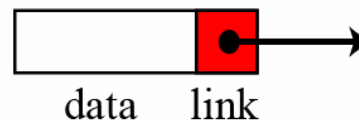
- Figure shows a linked list called *scores* that contains four elements and an empty linked list to be only a null pointer:



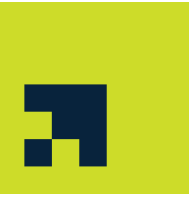
An empty
linked list



A node



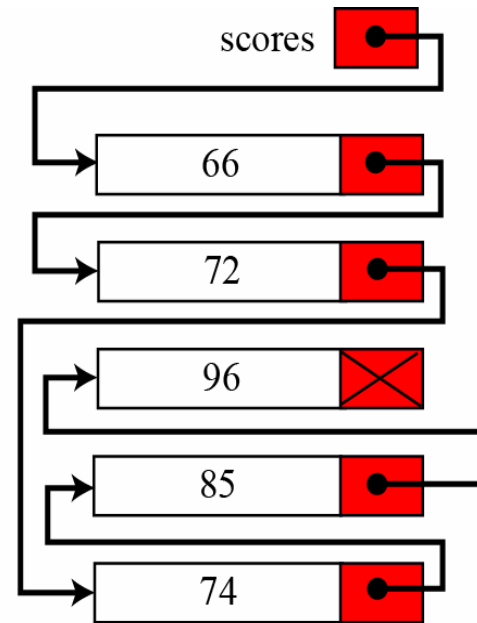
Arrays versus linked lists



- Both an array and a linked list are representations of a list of items in memory.
- The only difference is the way in which the items are linked together.
- Figure compares the two representations for a list of five integers.

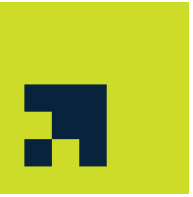
	scores
scores [1]	66
scores [2]	72
scores [3]	74
scores [4]	85
scores [5]	96

a. Array representation

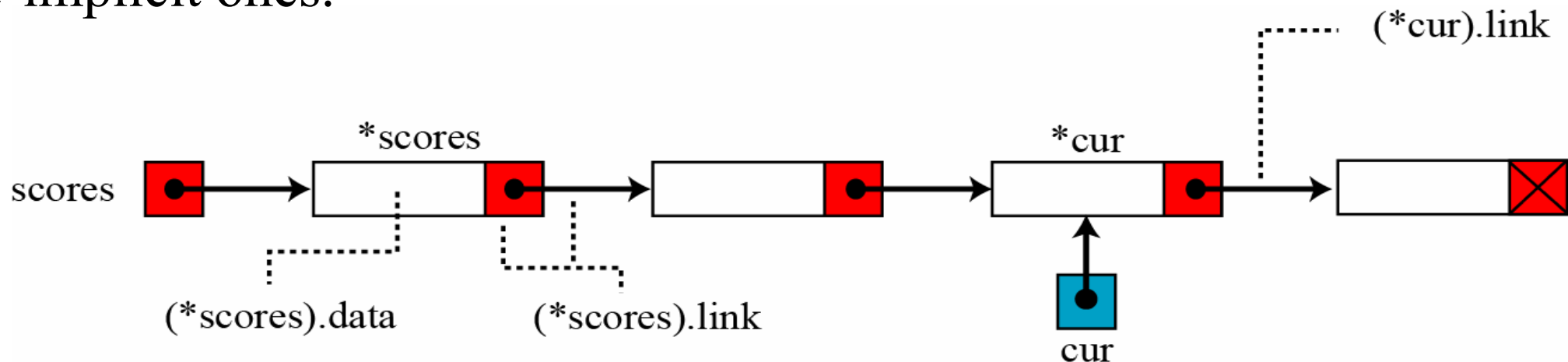


b. Linked list representation

Linked list names versus nodes names



- A linked list must have a name.
 - The name of a linked list is the name of the head pointer that points to the first node of the list.
- Nodes, on the other hand, do not have an explicit names in a linked list, just implicit ones.



Operations on linked lists



- The same operations we defined for an array can be applied to a linked list.
- *Searching a linked list*
- *Inserting*
- *Deleting*
- *Traversing a linked list*

Applications of linked lists

- A linked list is a very efficient data structure for sorted list that will go through many insertions and deletions.
- A linked list is a dynamic data structure in which the list can start with no nodes and then grow as new nodes are needed.
- A node can be easily deleted without moving other nodes, as would be the case with an array.
- For example, a linked list could be used to hold the records of students in a school. Each quarter or semester, new students enroll in the school and some students leave or graduate.

A linked list is a suitable structure if a large number of insertions and deletions are needed, but searching a linked list is slower than searching an array.



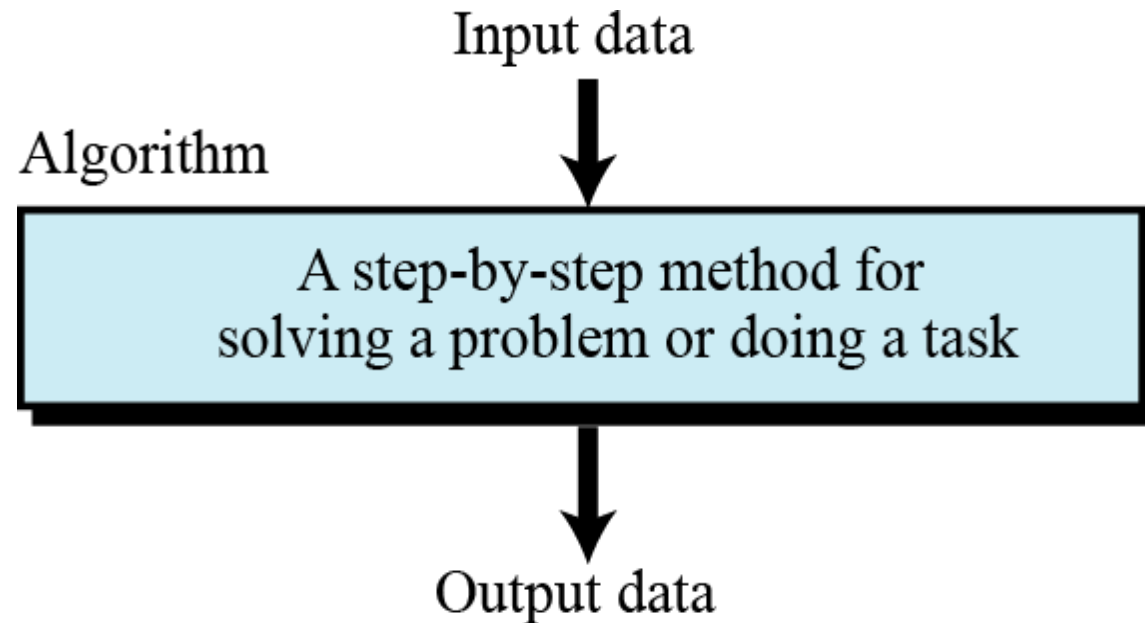
Algorithm

Algorithm



An informal definition of an algorithm is:

a step-by-step method for solving a problem or doing a task

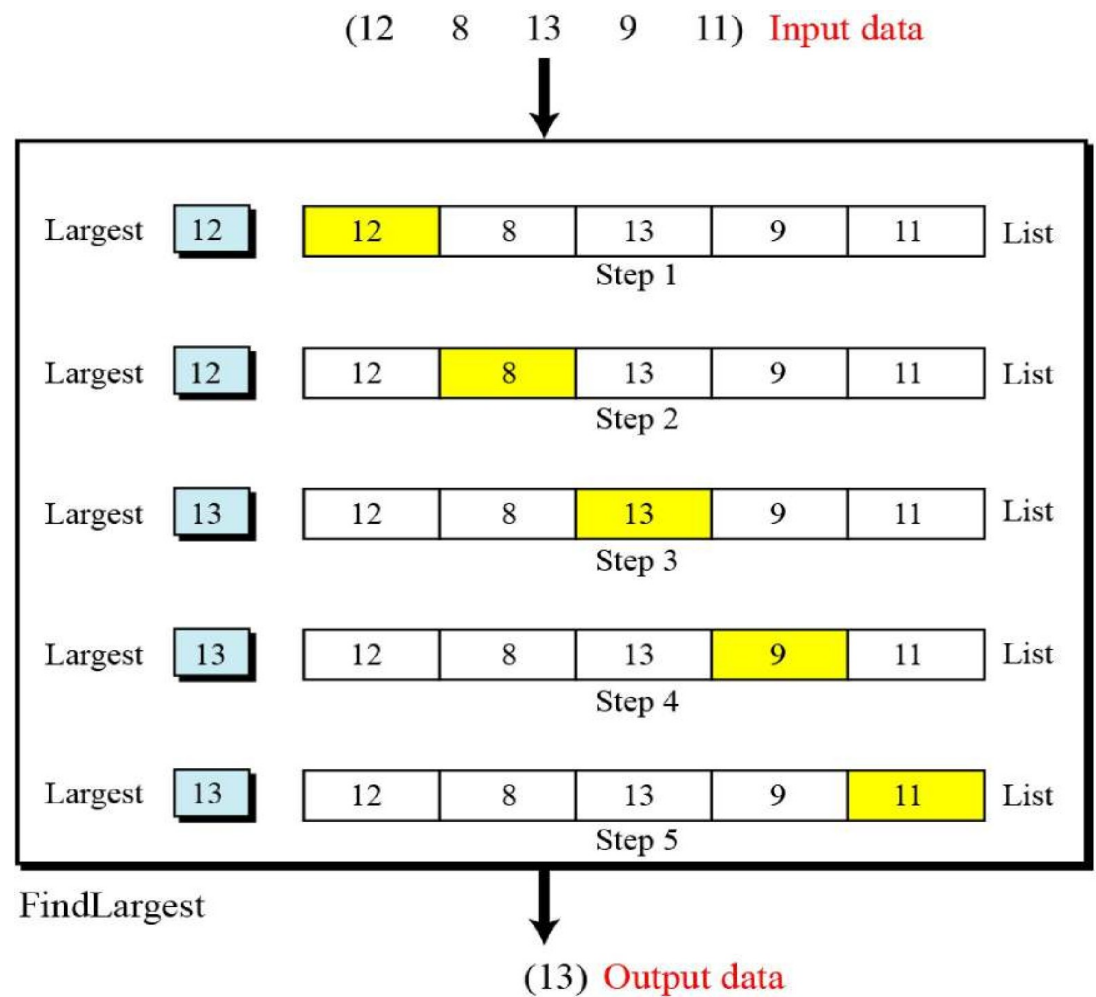


Example



- We want to develop an algorithm for finding the largest integer among a list of positive integers. The algorithm should find the largest integer among a list of any values (for example 5, 1000, 10,000, 1,000,000). The algorithm should be general and not depend on the number of integers.
- To solve this problem, we need an intuitive approach. First use a small number of integers (for example, five), then extend the solution to any number of integers.

Figure 8.2 shows one way to solve this problem. We call the algorithm FindLargest. Each algorithm has a name to distinguish it from other algorithms. The algorithm receives a list of five integers as input and gives the largest integer as output.

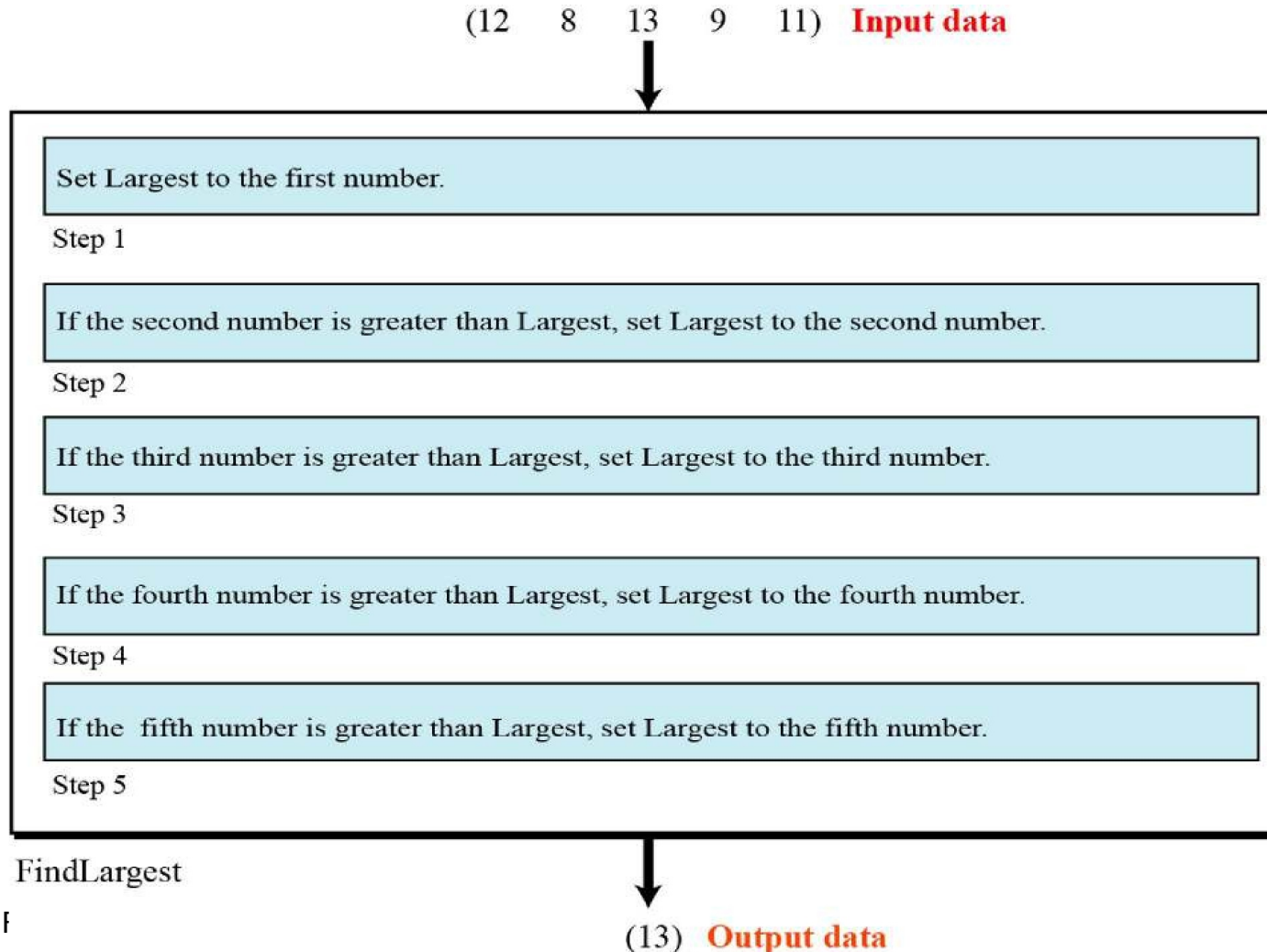


Finding the largest integer among five integers

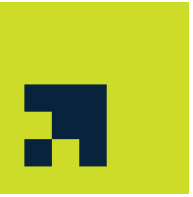
Defining actions



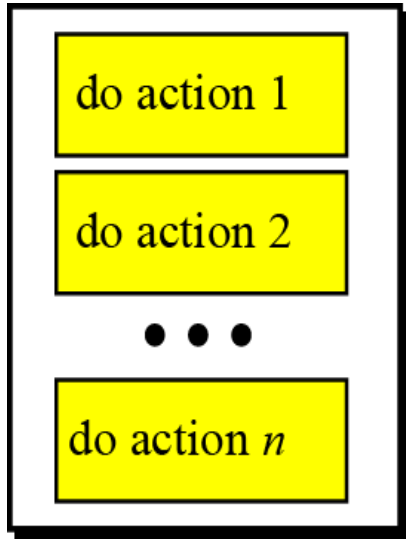
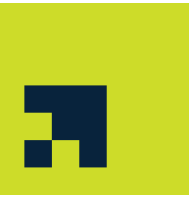
The previous figure does not show what should be done in each step. We can modify the figure to show more details.



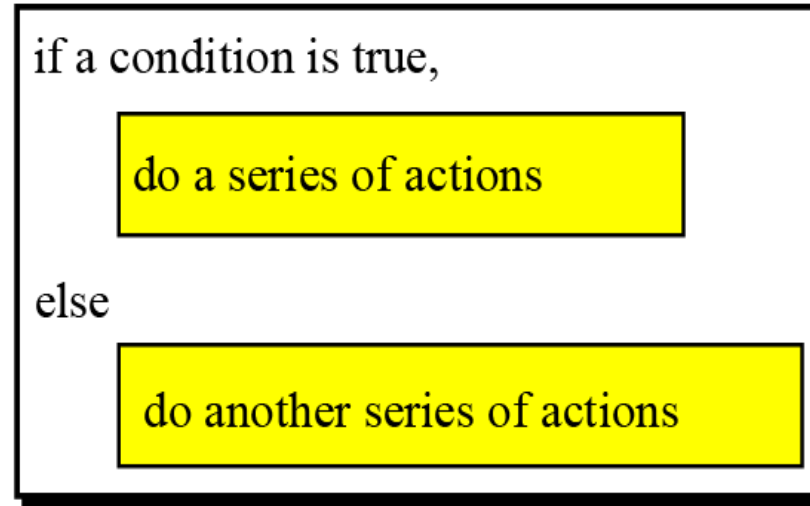
THREE CONSTRUCTS



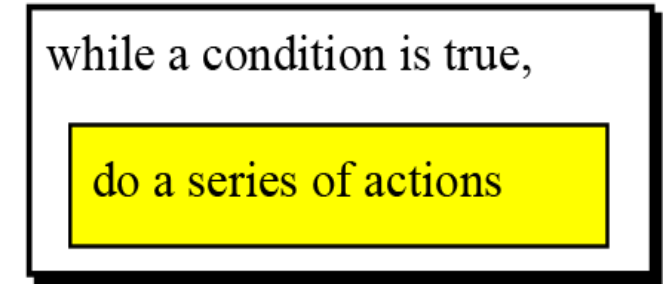
- Computer scientists have defined **three constructs** for a structured program or algorithm.
- The idea is that a program must be made of a combination of only these three constructs:
 - *sequence*
 - *decision* (selection)
 - *repetition*
- It has been proven there is no need for any other constructs.
- Using only these constructs makes a program or an algorithm easy to understand, debug or change.



a. Sequence



b. Decision



c. Repetition

Three constructs

Sequence

An algorithm, and eventually a program, is a sequence of instructions, which can be a simple instruction or either of the other two constructs.

Decision

Some problems cannot be solved with only a sequence of simple instructions. Sometimes we need to test a condition. If the result of testing is true, we follow a sequence of instructions: if it is false, we follow a different sequence of instructions. This is called the decision (selection) construct.

Repetition

In some problems, the same sequence of instructions must be repeated. We handle this with the repetition or *loop* construct. Finding the largest integer among a set of integers can use a construct of this kind.

ALGORITHM REPRESENTATION

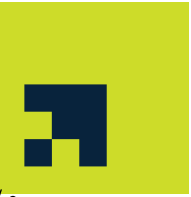


- Two of these tools:
 - *UML*
 - *pseudocode*

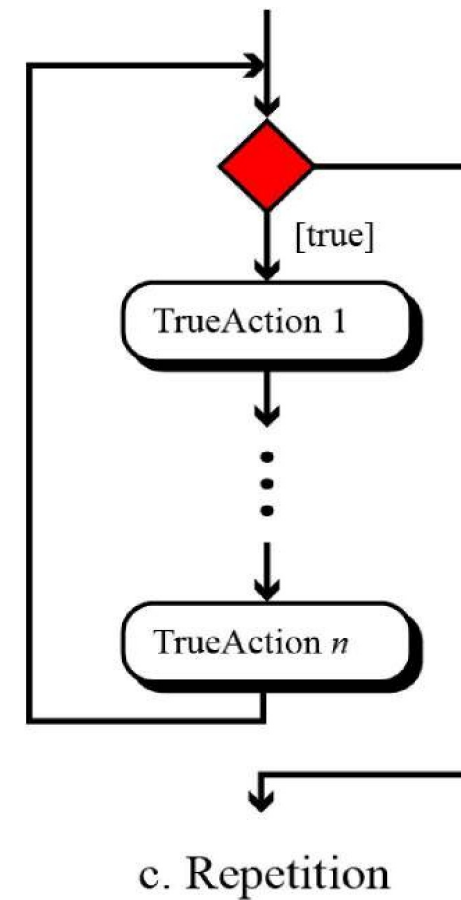
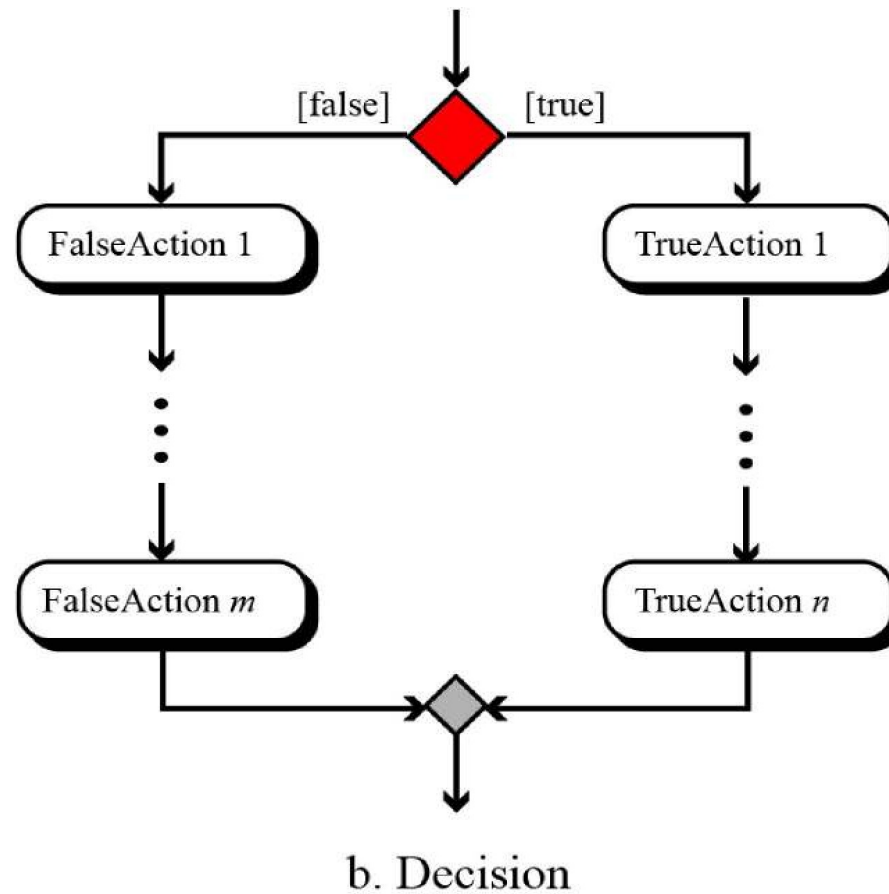
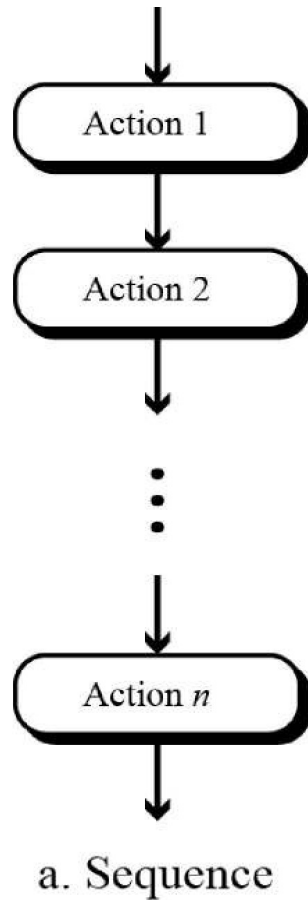
UML



- Unified Modeling Language (UML) is a pictorial representation of an algorithm.
- It hides all the details of an algorithm in an attempt to give the “big picture” and to show how the algorithm flows from beginning to end.



Here we show only how the three constructs are represented using UML.

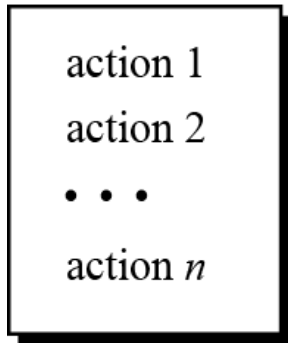


UML for three constructs

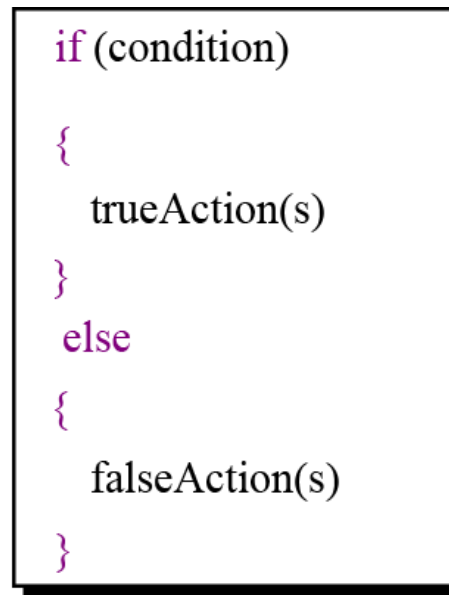
Pseudocode



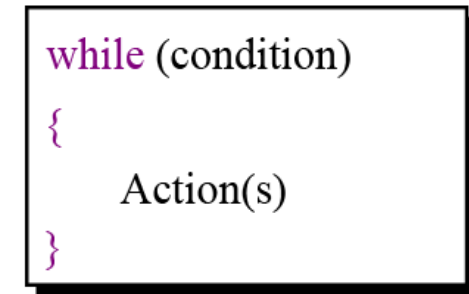
- Pseudocode is an English-language-like representation of an algorithm.
- No standard for pseudocode—some people use a lot of detail, others use less.
- Some use a code that is close to English, while others use a syntax like the Pascal programming language.



a. Sequence



b. Decision



c. Repetition

Example



Write an algorithm in pseudocode that finds the sum of two integers.

Algorithm 8.1 Calculating the sum of two integers

Algorithm: SumOfTwo (first, second)

Purpose: Find the sum of two integers

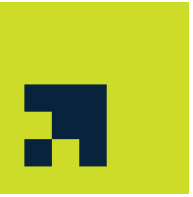
Pre: Given: two integers (first and second)

Post: None

Return: The sum value

```
{  
    sum ← first + second  
    return sum  
}
```

Example II



Write an algorithm to change a numeric grade to a pass/no pass grade.

Algorithm 8.2 Assigning pass / no pass grade

Algorithm: Pass/**NoPass** (score)

Purpose: Creates a pass/no pass grade given the score

Pre: Given: the score to be changed to grade

Post: None

Return: The grade

```
{  
    if (score  $\geq$  70)           grade  $\leftarrow$  "pass"  
    else                       grade  $\leftarrow$  "nopass"  
    return grade  
}
```

Example III



Write an algorithm to find the largest of a set of integers. We do not know the number of integers.

Algorithm 8.4 Finding the largest integer among a set of integers

Algorithm: FindLargest (list)

Purpose: Find the largest integer among a set of integers

Pre: Given: the set of integers

Post: None

Return: The largest integer

```
{  
    largest  $\leftarrow -\infty$   
    while (more integers to check)  
    {  
        current  $\leftarrow$  next integer  
        if (current > largest)      largest  $\leftarrow$  current  
    }  
    return largest  
}
```

FORMAL DEFINITION



An ordered set of unambiguous steps that produces a result and terminates in a finite time.

Ordered set

An algorithm must be a well-defined ordered set of instructions.



Unambiguous steps

Each step in an algorithm must be clearly and unambiguously defined. If one step is to add two integers, we must define both “integers” as well as the “add” operation: we cannot for example use the same symbol to mean addition in one place and multiplication somewhere else.

Produce a result

An algorithm must produce a result, otherwise it is useless. The result can be data returned to the calling algorithm, or some other effect (for example, printing).

Terminate in a finite time

An algorithm must terminate (halt). If it does not (that is, it has an infinite loop), we have not created an algorithm.

BASIC ALGORITHMS

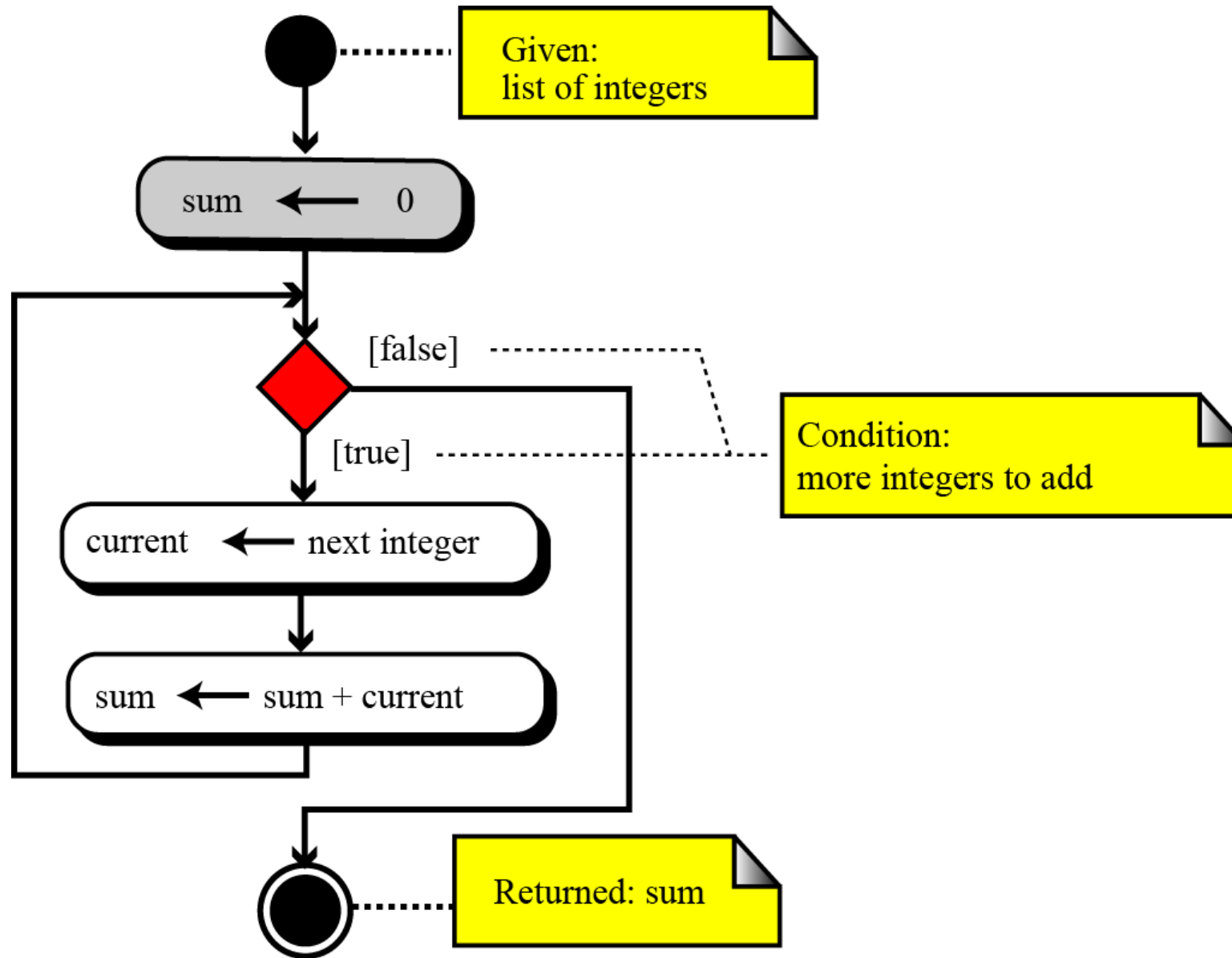


Summation

One commonly used algorithm in computer science is **summation**. We can add two or three integers very easily, but how can we add many integers? The solution is simple: we use the add operator in a loop.

A summation algorithm has three logical parts:

1. Initialization of the sum at the beginning.
2. The loop, which in each iteration adds a new integer to the sum.
3. Return of the result after exiting from the loop.



Summation algorithm

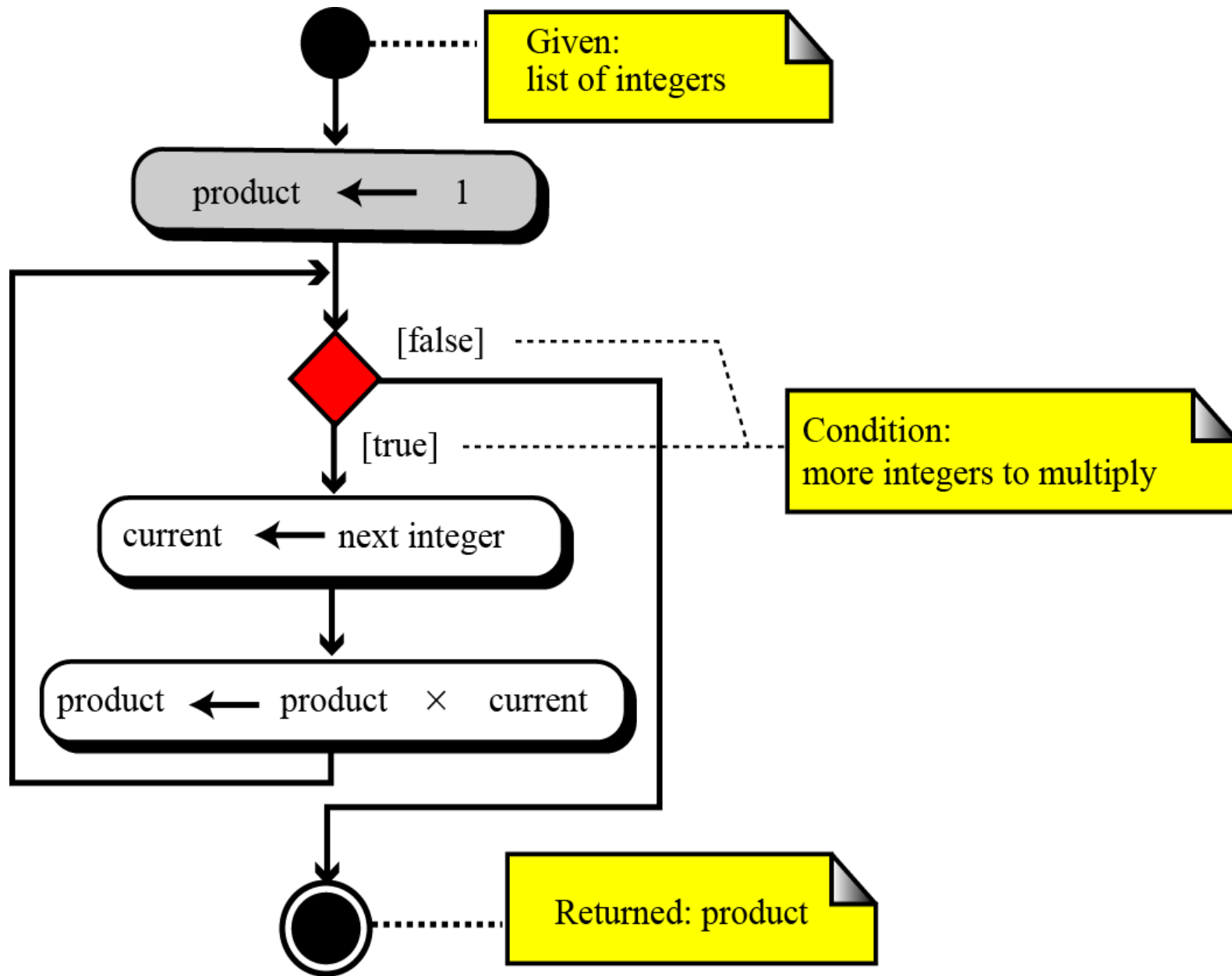
Product



Another common algorithm is finding the product of a list of integers. The solution is simple: use the multiplication operator in a loop.

A product algorithm has three logical parts:

1. Initialization of the product at the beginning.
2. The loop, which in each iteration multiplies a new integer with the product.
3. Return of the result after exiting from the loop.

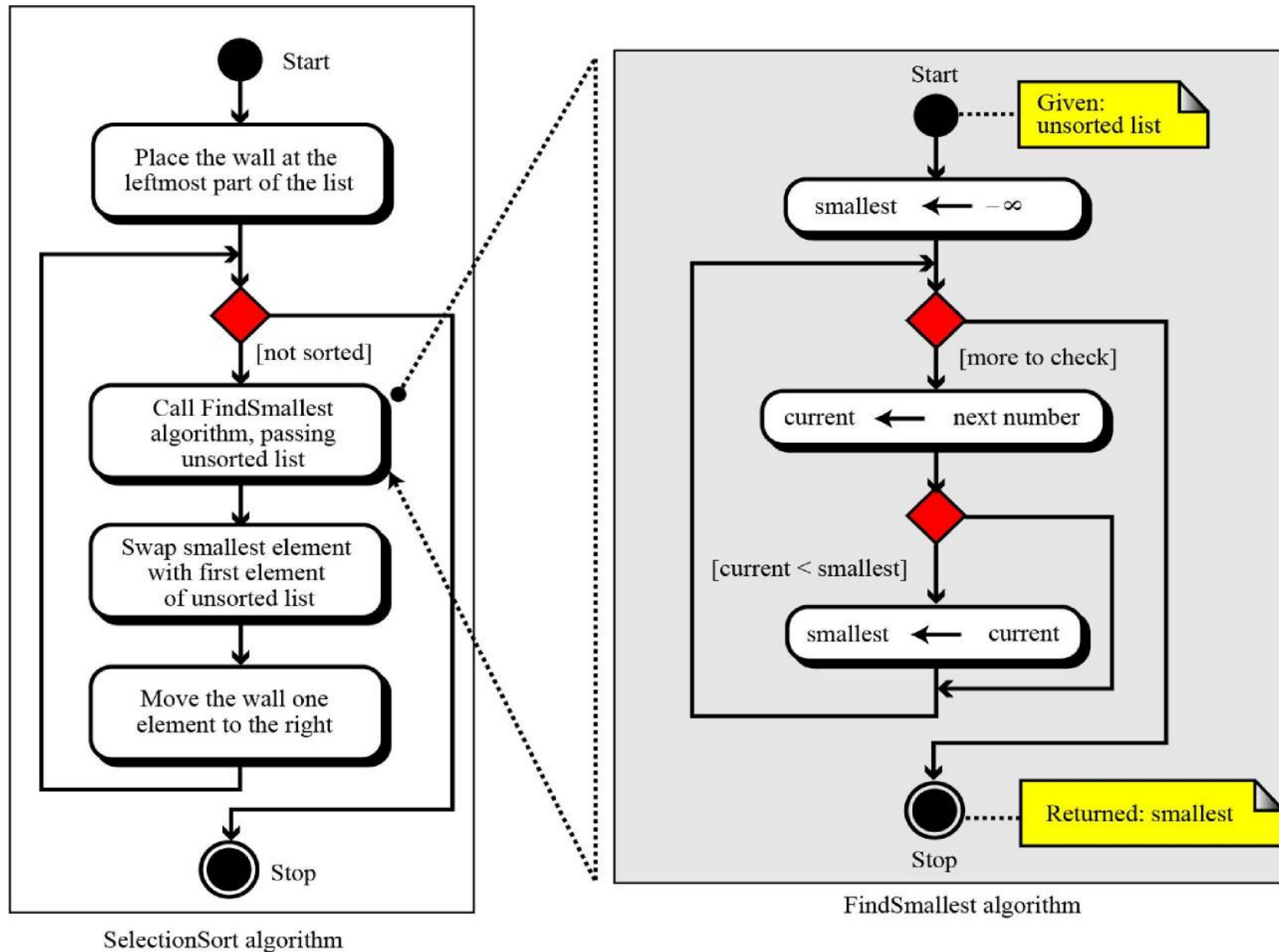


Product algorithm

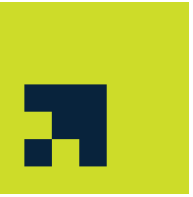
SUBALGORITHMS



- The principles of structured programming, however, require that an algorithm be broken into small units called **subalgorithms**.
- Each subalgorithm is in turn divided into smaller subalgorithms.



RECURSION



- In general, there are two approaches to writing algorithms for solving a problem.
- One uses **iteration**, the other uses **recursion**.
- Recursion is a process in which an algorithm calls itself.
- A recursive procedure *must* have the following *two properties*:
 - There must be certain criteria, called base criteria, for which the procedure does not call itself.
 - Each time the procedure does call itself, it must be closer to the base criteria.

Iterative definition



- Consider the calculation of a factorial.
- The factorial of a integer is the product of the integral values from 1 to the integer.
- The definition is iterative.

$$\text{Factorial } (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n - 1) \times (n - 2) \cdots 3 \times 2 \times 1 & \text{if } n > 0 \end{cases}$$

Recursive definition



An algorithm is defined recursively whenever the algorithm appears within the definition itself.

$$\text{Factorial } (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times \text{Factorial } (n - 1) & \text{if } n > 0 \end{cases}$$



Let us calculate $4!$ using the recursive definition. This calculation requires the following nine steps:

$$(1) \quad 4! = 4 \cdot 3!$$

$$(2) \quad 3! = 3 \cdot 2!$$

$$(3) \quad 2! = 2 \cdot 1!$$

$$(4) \quad 1! = 1 \cdot 0!$$

$$(5) \quad 0! = 1$$

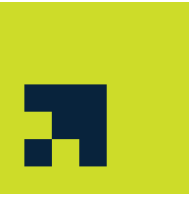
$$(6) \quad 1! = 1 \cdot 1 = 1$$

$$(7) \quad 2! = 2 \cdot 1 = 2$$

$$(8) \quad 3! = 3 \cdot 2 = 6$$

$$(9) \quad 4! = 4 \cdot 6 = 24$$

Iterative solution



This solution usually involves a loop.

Algorithm: Factorial (n)

Purpose: Find the factorial of a number using a loop

Pre: Given: n

Post: None

Return: $n!$

```
{  
     $F \leftarrow 1$   
     $i \leftarrow 1$   
    while ( $i \leq n$ )  
    {  
         $F \leftarrow F \times i$   
         $i \leftarrow i + 1$   
    }  
    return  $F$   
}
```

Recursive solution



The recursive solution does not need a loop, as the recursion concept itself involves repetition.

Algorithm 8.7 Pseudocode for recursive solution of factorial problem

Algorithm: Factorial (n)

Purpose: Find the factorial of a number using recursion

Pre: Given: n

Post: None

Return: $n!$

```
{  
    if ( $n = 0$ )        return 1  
    else              return  $n \times \text{Factorial}(n - 1)$   
}
```